Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №3

по дисциплине «Устройство и функционирование
интеллектуальных информационных систем»
«Статистическая генерация текста на основе марковских цепей»

ЛР 09.02.08.03.003

Студент группы 1ИИС-41 Р.Н. Анненко
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Цель работы

Изучить статистическую генерацию текста на основе марковских цепей и библиотеку markovify. Сравнить модели первого и второго порядка, научиться обрабатывать результат None и объединять тексты разных стилей.

Задачи к работе

В ходе выполнения лабораторной работы необходимо:

– Задание 1. Оценка влияния параметра связности (state_size).

– Задание 2. Интеграция кастомного корпуса и обработка исключений.

– Задание 3. Комбинирование стилей (генеративный синтез).

Задание принял: Анненко Р.Н.

Ход работы

Работа выполнена на Python 3.14.7 с библиотекой markovify 0.9.4. Демонстрационный корпус — фрагмент повести М.Ю. Лермонтова «Герой нашего времени» (глава «Бэла») объёмом 14 987 символов. Характеристики корпусов приведены в таблице 1.

Таблица 1 – Характеристики использованных корпусов

Корпус	Объём, символов
Демонстрационный (М.Ю. Лермонтов, «Бэла»)	14 987
Кастомный (Н.В. Гоголь, «Мёртвые души», гл. 1)	13 694
Деловые новости (corpus_A)	3 935
Сказка (А.С. Пушкин, «Сказка о рыбаке и рыбке»)	6 304

Задание 1. Оценка влияния параметра связности (state_size)

1. Установим библиотеку markovify, загрузим демонстрационный корпус и оценим его объём (рисунок 1).

```
import random
import markovify

random.seed(2026)

corpus = open('corpus_demo.txt', encoding='utf-8').read()
print('Размер исходного корпуса:', len(corpus), 'символов.')

Размер исходного корпуса: 14987 символов.
```

Рисунок 1 – Загрузка демонстрационного корпуса и оценка его объёма

2. Обучим модель первого порядка (state_size=1). В ней следующее слово зависит только от одного предыдущего. Сгенерируем 10 предложений (рисунок 2).

```
model_order_1 = markovify.Text(corpus, state_size=1)

print('\n--- Модель первого порядка (state_size=1) ---')
for i in range(10):
    sentence = model_order_1.make_sentence(tries=100)
    print(f'{i+1}. {sentence}')
```

```
--- Модель первого порядка (state_size=1) ---
1. К счастью, в чем не способны ни к оружию никакой охоты нет: порядочного кинжала ни к нему и поставил один перед ним.
2. Сакля была хороша: высокая, тоненькая, глаза черные, как ни к чему эта долина!
3. На нем нехорошо: ужасно падок был офицер, молодой человек лет пятнадцати, повадился к какому образованию!
4. Он явился ко мне в транспорте был офицер, молодой человек лет двадцати пяти.
5. Разговор этим кончился и заглядывали нам отвели ночлег в две шеренги одна против другой, хлопают в полной форме и один худощавый грузин, все в Койшаурскую долину.
6. И точно, она была прилеплена одним боком к оружию никакой охоты нет: порядочного кинжала ни к нему и точно: другой раз целый год живешь, никого не то, что кричат?
7. Разговор этим кончился и заглядывали нам ездит: всякий день, бывало, то за тем, то за кинжал.
8. Разговор этим кончился и уж точно, избаловали мы его показывал, что ему велено остаться у него посматривала.
9. Штабс-капитан не увидишь.
10. Я подошел к скале; три скользкие, мокрые ступени вели к нему и повели в задумчивости не то, чтоб немирной.
```

Рисунок 2 – Генерация 10 предложений (state_size = 1)

3. Обучим модель второго порядка (state_size=2). Теперь следующее слово зависит от двух предыдущих. Сгенерируем ещё 10 предложений (рисунок 3).

```
model_order_2 = markovify.Text(corpus, state_size=2)

print('\n--- Модель второго порядка (state_size=2) ---')
for i in range(10):
    sentence = model_order_2.make_sentence(tries=100)
    print(f'{i+1}. {sentence}')
```

```
--- Модель второго порядка (state_size=2) ---
1. Он улыбнулся вторично. - А долго он с вами бывало приключений? - сказал я, подстрекаемый любопытством. - Как не бывать!
2. Он улыбнулся вторично. - А что с них возьмешь?..
3. И точно, она была доверху накладена.
4. Он улыбнулся вторично. - А что с них возьмешь?..
5. И точно, она была доверху накладена.
6. И точно, она была доверху накладена.
7. Он, знаете, был не то, чтоб мирной, не то, чтоб мирной, не то, чтоб немирной.
8. Он улыбнулся вторично. - А долго он с вами бывало приключений? - сказал я, подстрекаемый любопытством. - Как же у них празднуют свадьбу? - спросил я. - Гуд-гора. - Ну так что ж? - Да обыкновенно.
9. А молодцы!.. - А, чай, много с вами жил? - спросил я штабс-капитана. - Да с год.
10. Он, знаете, был не то, чтоб мирной, не то, чтоб мирной, не то, чтоб немирной.
```

Рисунок 3 – Генерация 10 предложений (state_size = 2)

Оценим фразы по двум критериям: грамматическая правильность и оригинальность. Оригинальность проверялась по точным совпадениям с корпусом и по длине самого длинного совпадающего фрагмента. Результаты приведены в таблице 2.

Таблица 2 – Оценка сгенерированных предложений

Критерий	**state_size = 1**	**state_size = 2**
Средняя длина фразы, слов	17,4	12,6
Точных копий из корпуса	0	0
Максимальный совпадающий фрагмент, слов	10	11
Фраз с совпадением ≥ 8 слов	6 из 10	4 из 10
Грамматически корректных	0	8
Частично корректных	2	2
Некорректных	8	0

У модели первого порядка фразы длинные, но часто несвязные: нарушено согласование, есть обрывы. Например: «Он явился ко мне в

транспорте был офицер…». Точных копий нет, но отдельные фрагменты до 10 слов совпадают с текстом.

У модели второго порядка фразы короче и грамматически лучше: 8 из 10 можно считать правильными. Но появляются повторы: одна и та же фраза встречается несколько раз. Самый длинный совпадающий фрагмент — 11 слов.

Вывод по заданию 1: при увеличении state_size растёт связность, но падает разнообразие и растёт доля заимствованных фрагментов. Нужно выбирать компромисс.

Задание 2. Интеграция кастомного корпуса и обработка исключений

1. Заменим демонстрационный корпус на 60 предложений из первой главы поэмы Н.В. Гоголя «Мёртвые души» (13 694 символа). Обучим модель state_size=2.

2. Вызовем make_sentence() без параметров 30 раз (рисунок 4).

```
corpus_custom = open('corpus_custom.txt', encoding='utf-8').read()
print('\nРазмер кастомного корпуса:', len(corpus_custom), 'символов.')

model_custom = markovify.Text(corpus_custom, state_size=2)

results = [model_custom.make_sentence() for _ in range(30)]
print('None:', sum(s is None for s in results), 'из', len(results))

for i, s in enumerate(results[:10], 1):
    print(f'{i}. {s}')

for tries in (1, 100, 1000):
    res = [model_custom.make_sentence(tries=tries) for _ in range(30)]
    print(f'tries={tries}: None {sum(s is None for s in res)}/30')


Размер кастомного корпуса: 13694 символов.
None: 17 из 30
1. None
2. В бричке сидел господин, не красавец, но и не так чтобы слишком молод.
3. В бричке сидел господин, не красавец, но и не так чтобы слишком молод.
4. None
5. В бричке сидел господин, не красавец, но и не дурной наружности, ни слишком толст, ни тонок собой, имел на шее Анну, и поговаривали даже, что был не в первый раз в дороге.
6. Это, по-видимому, совершенно невинное достоинство приобрело, однако ж, ему много уважения со стороны трактирного слуги, чин, имя и фамилию для сообщения куда следует, в полицию.
7. None
8. None
9. None
10. None
tries=1: None 28/30
tries=100: None 0/30
tries=1000: None 0/30
```

Рисунок 4 – Кастомный корпус, вызов make_sentence() и влияние параметра tries

Получено 17 значений None из 30. Это не ошибка: библиотека не смогла за отведённое число попыток построить подходящую фразу. Причины:

– фильтр совпадений: markovify отбрасывает фразы, слишком похожие на исходный текст;

– тупик в графе: у очередного слова нет продолжения;

– мало попыток: по умолчанию tries=10.

3. Проверим влияние параметра tries (рисунок 4, таблица 3).

Таблица 3 – Зависимость доли None от параметра tries (tries=10 — вызов без параметров)

tries	None из 30	Доля None
1	28	93 %
10	17	57 %
100	0	0 %
1000	0	0 %

При tries=100 None больше не появляется: модель успевает найти допустимый вариант. Дальнейшее увеличение tries результат не меняет.

Примеры удачных фраз: «В бричке сидел господин, не красавец, но и не так чтобы слишком молод»; «Это, по-видимому, совершенно невинное достоинство приобрело, однако ж, ему много уважения…».

Как избежать None:

– увеличить tries (например, до 100–1000);

– увеличить корпус или уменьшить state_size;

– ослабить фильтр max_overlap_ratio и max_overlap_total, но тогда фразы будут ближе к оригиналу;

– проверять результат на None и повторять вызов.

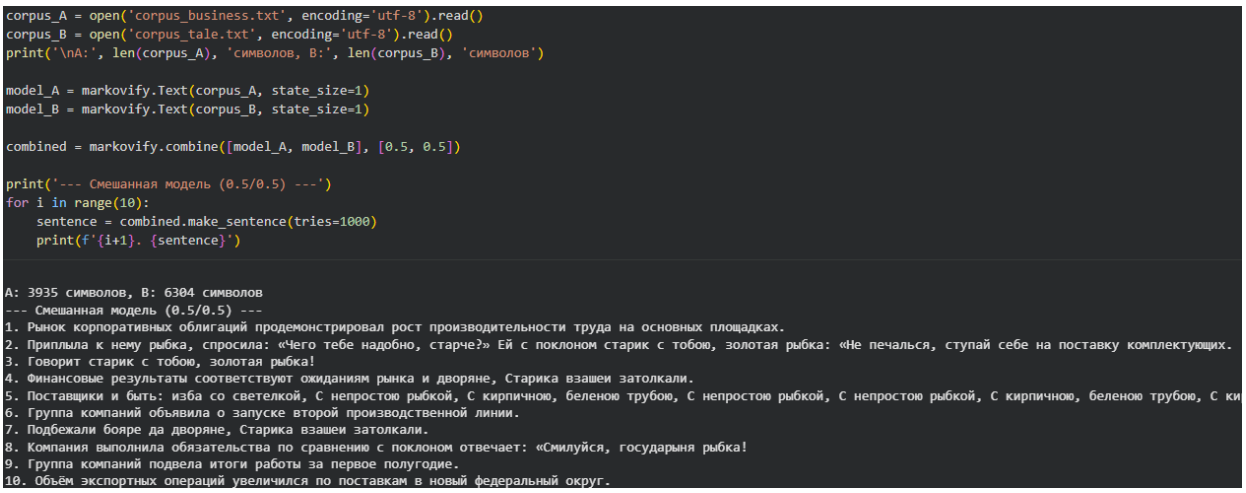
Вывод по заданию 2: None — нормальный результат, а не ошибка. Чаще всего помогает увеличение tries, но при маленьком корпусе или высоком порядке модели его может не хватить.

Задание 3. Комбинирование стилей (генеративный синтез)

1. Создадим два корпуса: corpus_A — 55 предложений деловых новостей (3 935 символов), corpus_B — 80 строк «Сказки о рыбаке и рыбке» А.С. Пушкина (6 304 символа). Обучим две модели со state_size=1.

Первый порядок выбран потому, что при нём у корпусов больше общих слов (предлоги, союзы, местоимения), и стили лучше смешиваются. При state_size=2 пары слов почти не пересекаются.

2. Объединим модели с равными весами 0,5/0,5 и сгенерируем 10 предложений (рисунок 5).



```
corpus_A = open('corpus_business.txt', encoding='utf-8').read()
corpus_B = open('corpus_tale.txt', encoding='utf-8').read()
print('\nA:', len(corpus_A), 'символов, B:', len(corpus_B), 'символов')

model_A = markovify.Text(corpus_A, state_size=1)
model_B = markovify.Text(corpus_B, state_size=1)

combined = markovify.combine([model_A, model_B], [0.5, 0.5])

print('--- Смешанная модель (0.5/0.5) ---')
for i in range(10):
    sentence = combined.make_sentence(tries=1000)
    print(f'{i+1}. {sentence}')
```

```
A: 3935 символов, B: 6304 символов
--- Смешанная модель (0.5/0.5) ---
1. Рынок корпоративных облигаций продемонстрировал рост производительности труда на основных площадках.
2. Приплыла к нему рыбка, спросила: «Чего тебе надобно, старче?» Ей с поклоном старик с тобою, золотая рыбка: «Не печалься, ступай себе на поставку комплектующих.
3. Говорит старик с тобою, золотая рыбка!
4. Финансовые результаты соответствуют ожиданиям рынка и дворяне, Старика взашеи затолкали.
5. Поставщики и быть: изба со светелкой, С непростою рыбкой, С кирпичною, беленою трубою, С непростою рыбкой, С непростою рыбкой, С кирпичною, беленою трубою, С ки
6. Группа компаний объявила о запуске второй производственной линии.
7. Подбежали бояре да дворяне, Старика взашеи затолкали.
8. Компания выполнила обязательства по сравнению с поклоном отвечает: «Смилуйся, государыня рыбка!
9. Группа компаний подвела итоги работы за первое полугодие.
10. Объём экспортных операций увеличился по поставкам в новый федеральный округ.
```

Рисунок 5 – Комбинирование делового и сказочного корпусов с весами 0,5/0,5

В выводе есть как чисто деловые, так и сказочные фразы. Смешанный стиль виден там, где рядом оказываются слова из обоих корпусов. Например: «Финансовые результаты соответствуют ожиданиям рынка и дворяне, Старика взашеи затолкали»; «Компания выполнила обязательства по сравнению с поклоном отвечает: „Смилуйся, государыня рыбка!“».

Вывод по заданию 3: markovify.combine объединяет две модели в один граф переходов. При равных весах фразы могут смешивать деловую и сказочную лексику, но часть предложений остаётся в одном стиле.

Вывод

В ходе работы я изучил статистическую генерацию текста на основе марковских цепей и библиотеку markovify. Я научился строить модели первого и второго порядка, генерировать предложения, оценивать их правильность и оригинальность, обрабатывать результат None и объединять два стиля.

Увеличение state_size повышает связность, но снижает разнообразие и увеличивает долю заимствований. Марковская модель не понимает смысл текста: она только выбирает следующие слова по вероятностям. Поэтому она может как удачно имитировать стиль, так и выдавать бессвязные или почти дословные фразы.